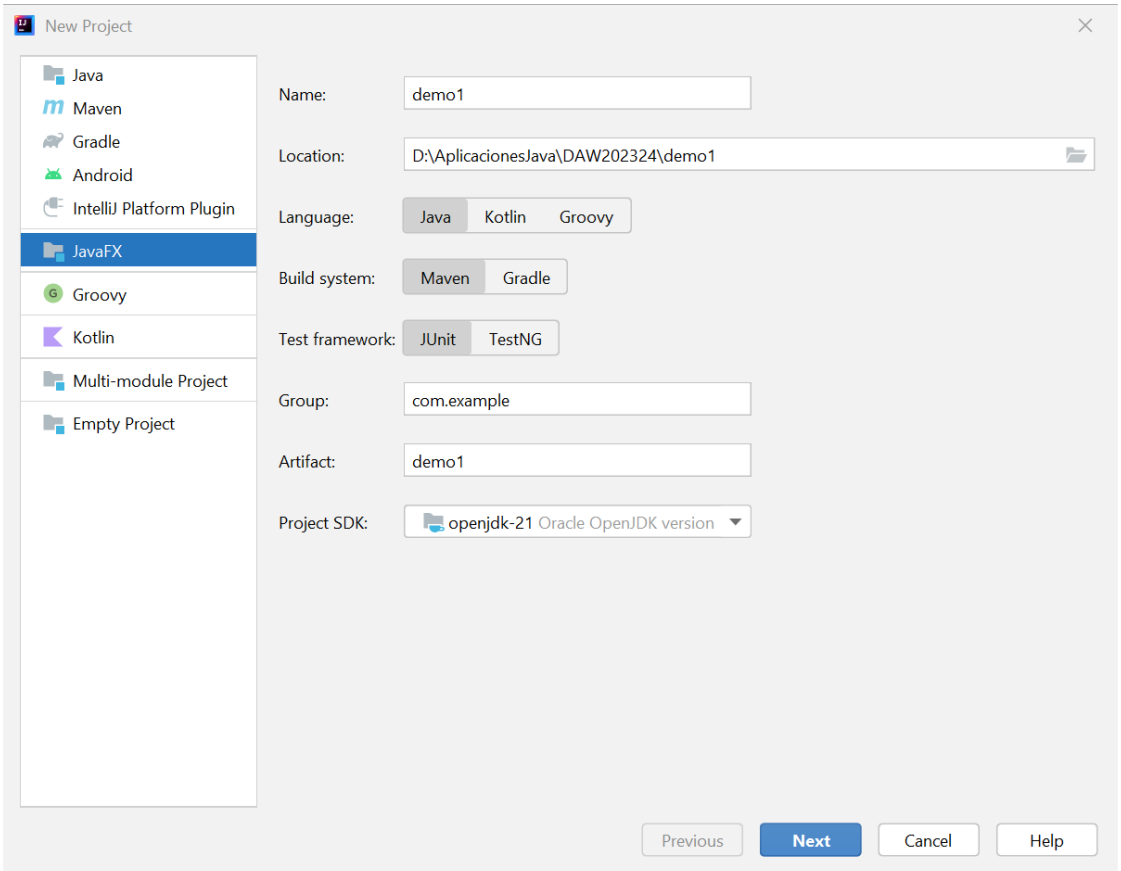
QUÉ ES JAVAFX

JavaFX es un conjunto de paquetes Java que nos permite crear una amplia variedad de interfaces gráficas de usuario (GUI), desde las clásicas con controles típicos como etiquetas, botones, cuadros de texto, menús, etc., hasta algunas aplicaciones avanzadas y modernas., con algunas opciones interesantes como animaciones o perspectiva.
Si miramos hacia atrás, podemos ver JavaFX como una evolución de una librería Java anterior, llamada Swing, que todavía está incluida en el JDK oficial, aunque está quedando bastante obsoleta y las posibilidades que ofrece son mucho más reducidas. Es por lo que ahora la mayoría de las aplicaciones de escritorio Java se desarrollan con JavaFX.

Crear un proyecto JavaFx

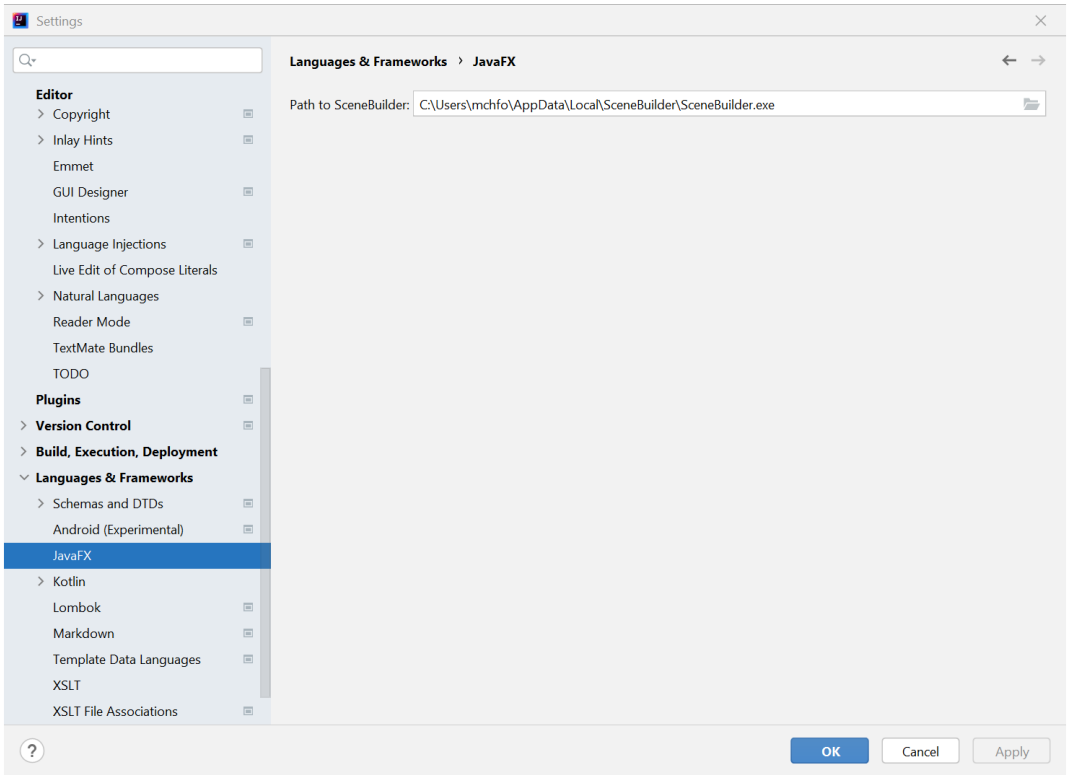
1. Crear un nuevo proyecto.

File → New Proyect



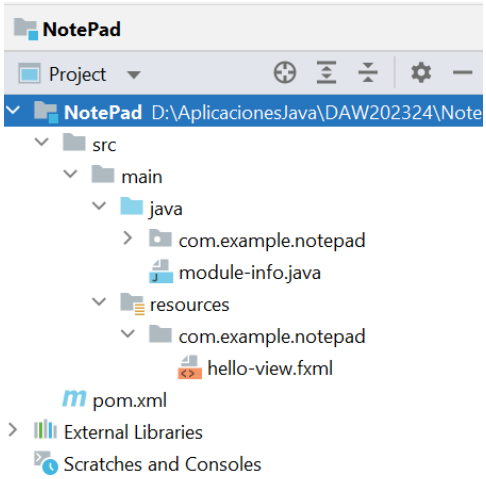
2. Vincular la aplicación con SceneBuilder

File → Settings →Languages & Framworks → Java FX



Ponemos la ruta del exe del SceneBuilder.

En el proyecto con Java 21 Se crea el siguiente sistema de directorios.



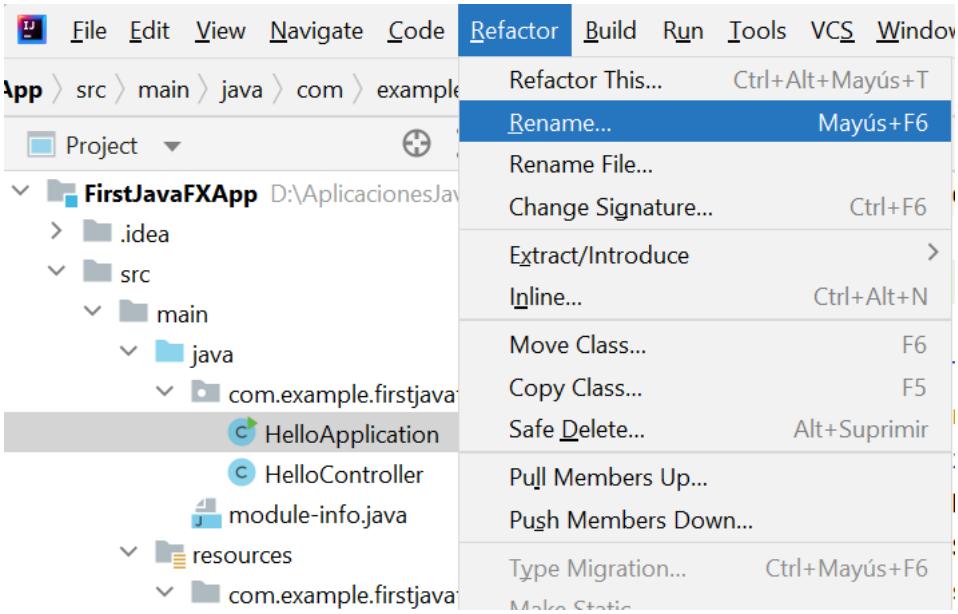
En la carpeta java, estará el código fuente del proyecto, en la carpeta resources, los fxml y las imágenes.

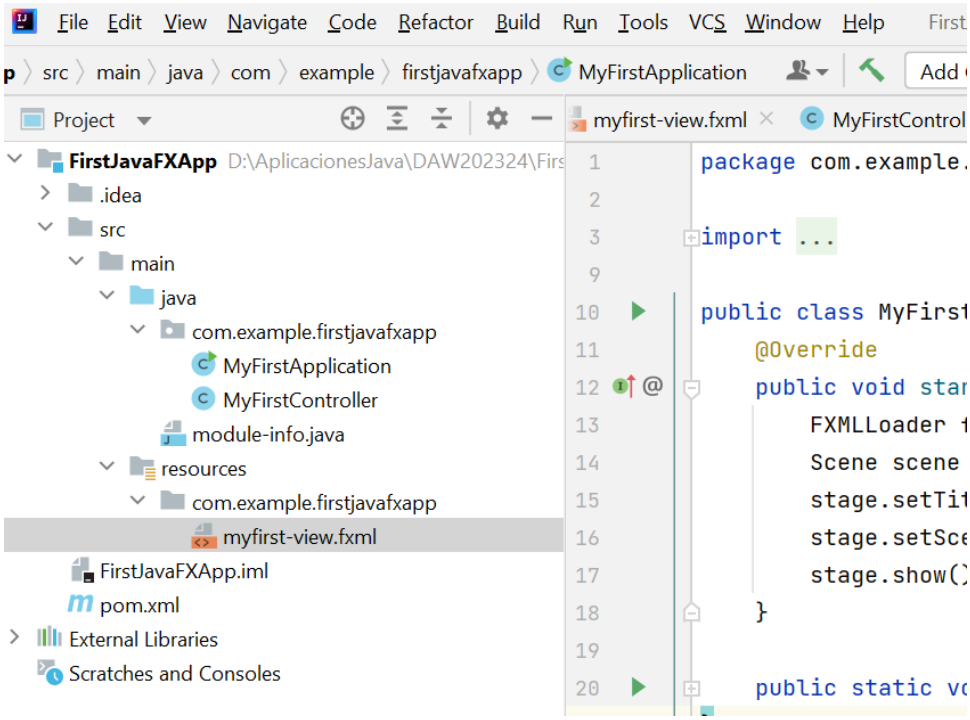
Dentro de la carpeta java, inicialmente habrá dos ficheros HelloAplication.java, es el fichero de arranque de la aplicación y HelloController.java es el fichero controlador, y está enlazado con el fxml, a través de esta sentencia

`fx:controller="com.example.notepad.HelloController"` en el hello-view.fxml

3. Renombrar ficheros.

A mí, personalmente, me gusta renombrar los ficheros que me crea el intellij y poner mis propios nombres.

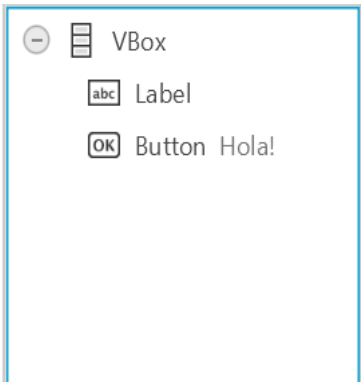




El controlador y las vistas

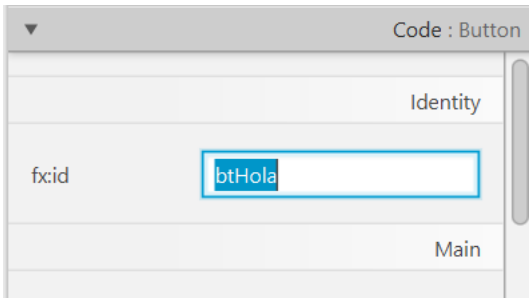
4. Diseñar mi primera App.

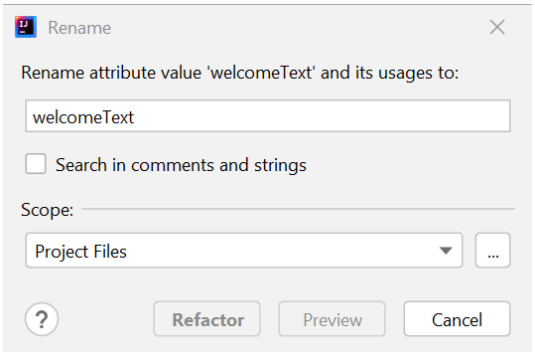
La aplicación por defecto, que crea el propio Intellij tiene estos componentes.



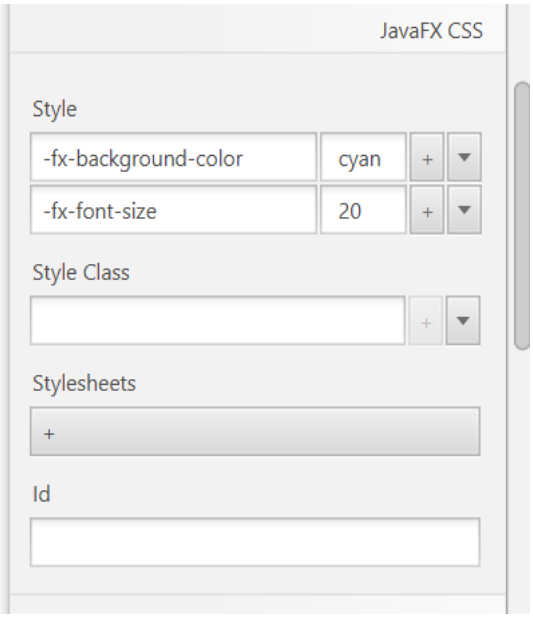

```
<VBox     alignment="CENTER"     spacing="20.0"
            xmlns:fx="http://javafx.com/fxml/1"
            xmlns="http://javafx.com/javafx/21"
fx:controller=     "com.example.firstjavafxapp.
MyFirstController">
      <padding>
      <Insets     bottom="20.0"     left="20.0"
right="20.0" top="20.0" />
                                          </padding>
<Label fx:id="welcomeText" />
      <Button    onAction="#onHelloButtonClick"
text="Hola!" />
</VBox>
```

Ahora le añadiremos un fx:id al botón, el label ya tiene uno por defecto, pero podemos modificarlo también si no nos gusta, siempre utilizaremos *refactor* para evitar errores con los cambios de nombre.





Si queremos, también podríamos añadir código CSS.



5. Crear los campos en el controlador.

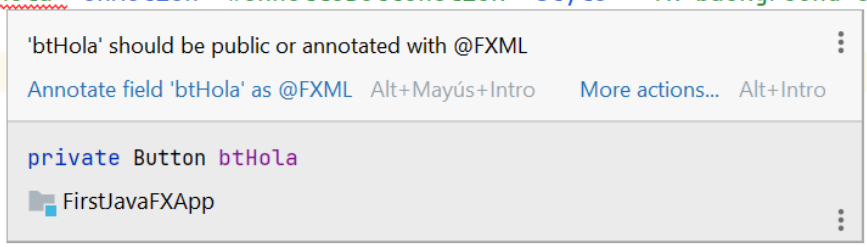
```
public class MyFirstController {
    @FXML
    private Label lbBienvenido; //Este ya estaba creado y solo lo he
renombrado

    private Button btHola; //Este lo he creado yo

    @FXML
    protected void onHelloButtonClick() {
        lbBienvenido.setText("Welcome to JavaFX Application!");
    }
}
```

Al crear los campos en el controlador en el fichero .fxml me da un error, el intellij lo solventa marcando los campos con una anotación (un decorador)

```
<Button fx:id="btHola" onAction="#onHelloButtonClick" style="-fx-background-c
Box>
```



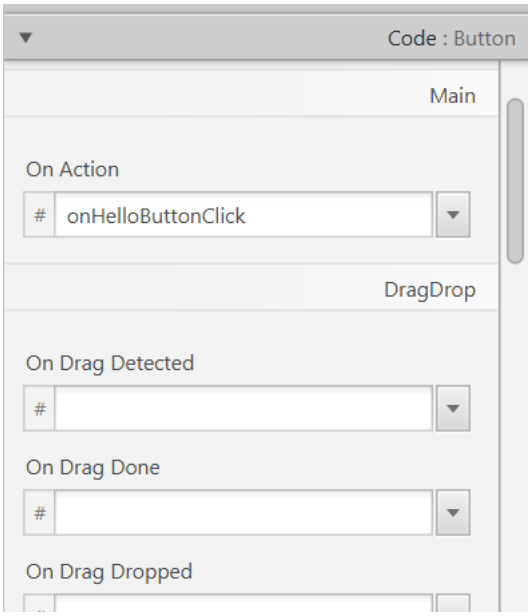

```
public class MyFirstController {
    @FXML
    private Label lbBienvenido; //Este ya estaba creado y solo lo he
renombrado
```

```
    @FXML
    private Button btHola; //Este lo he creado yo

    @FXML
    protected void onHelloButtonClick() {
        lbBienvenido.setText("Welcome to JavaFX Application!");
    }
}
```

6. Asignar acciones.

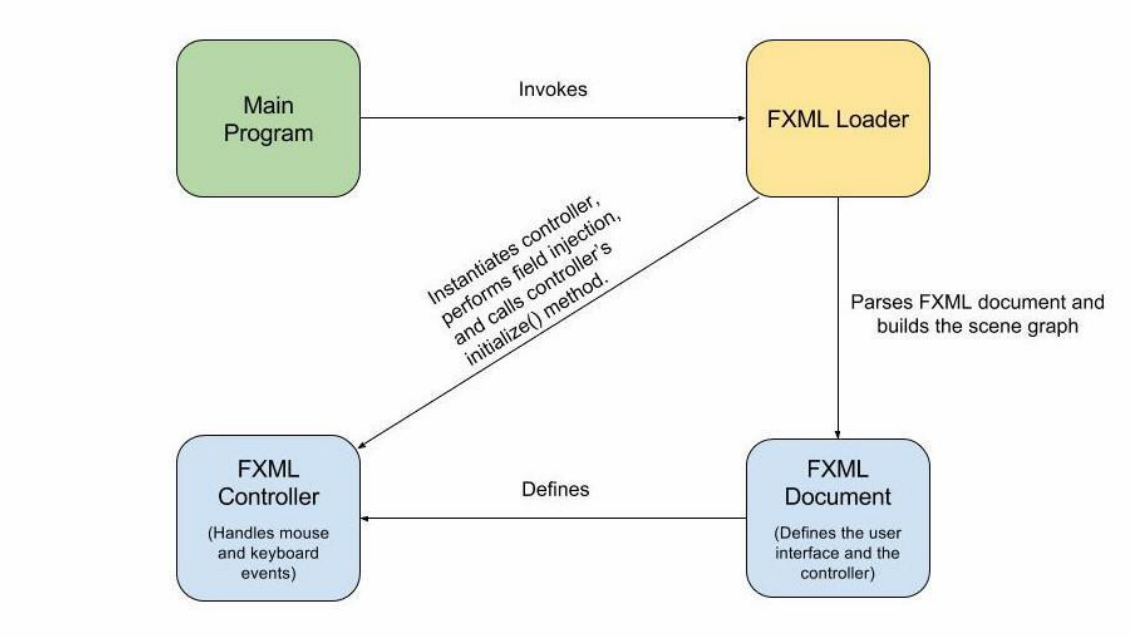
El botón creado por defecto tiene una acción asociada en el controlador.




```
@FXML
protected void onHelloButtonClick()
{
    lbBienvenido.setText("Bienvenido
a mi primera aplicación en Java
FX!");
}
```

Lo que hace es modificar el texto del label por ese nuevo texto que le pasamos al hacer Click en el botón.

7. Estructura de una aplicación JavaFX



La figura anterior muestra la estructura de una aplicación JavaFX FXML típica.

Como se muestra en la figura, la interfaz de usuario de una aplicación FXML se define dentro de un documento FXML y toda la lógica para manejar eventos de entrada se escribe dentro de una clase de controlador.

La ejecución del programa comienza con la clase Main, que invoca al loader FXML. El loader FXML analiza el documento FXML, crea instancias de los nodos especificados en el documento y crea el gráfico de escena.

Después de construir el gráfico de escena, el loader FXML crea una instancia de la clase de controlador, inyecta los campos definidos en la clase de controlador con objetos instanciados del documento fxml y luego llama al método inicializar() del controlador (se verá más adelante).

El Stage y la Escena

8. La clase Stage

Como hemos visto en la aplicación de muestra que nos crea el Intellij, la clase principal hereda de la clase Application

```
public class MyFirstApplication extends Application {
```

El objeto Stage es una referencia al contenedor principal de nuestra aplicación. Esta será una ventana en sistemas operativos como Linux, Windows o Mac OS X, pero puede ser la pantalla completa si nuestra aplicación se ejecuta en un *smartphone* o una *tablet*.

La clase Stage nos provee de muchos métodos útiles para cambiar características de la ventana principal. Algunos ejemplos son estos:

- ***setTitle(String)***: modifica el título de la aplicación es visible en la barra de arriba de la ventana.
- ***setScene(Scene):*** establece nuestra escena de aplicación (donde se colocarán todos los controles). Más adelante aprenderemos que puede haber más de una escena en un escenario.• show: makes the application (stage) visible, and keeps on running next instructions
- ***showAndWait***: hace que la aplicación (*stage*) sea visible y espera hasta que se cierre antes de continuar
- ***setMinWidth(double), setMaxWidth(double):*** establece el ancho mínimo y máximo (respectivamente) de la ventana, de modo que no podamos cambiar su tamaño más allá de estos límites.
- ***setMinHeight(double), setMaxHeight(double):*** establece el alto mínimo y máximo (respectivamente) de la ventana, de modo que no podamos cambiar su tamaño más allá de estos límites.
- ***getMinWidth, getMaxWidth, getMinHeight, getMaxHeight***: obtiene el máximo o mínimo ancho o alto de la aplicación

- ***setFullScreen(boolean):*** establece si nuestra aplicación se ejecutará en modo de pantalla completa (por lo que no se podrá cambiar su tamaño y no habrá ninguna barra superior), o no.

- ***setMaximized(boolean)***: establece si nuestra aplicación está maximizada o no.

- ***setIconified(boolean):*** establece si nuestra aplicación está iconificada (minimizada) o no.

- ***setResizable(boolean):*** establece si nuestra aplicación es redimensionable o no.

https://docs.oracle.com/javase/8/javafx/api/javafx/stage/Stage.html

9. La Escena

Todos los proyectos JavaFx deben tener al menos una escena, que contendrá todos los controles de la aplicación. Cuando lo creamos, necesitamos especificar su nodo principal (el que devuelve FXMLLoader cuando analizamos un FXML):

```
FXMLLoader fxmlLoader = new FXMLLoader
(HelloApplication.class.getResource( "hello-view.fxml" ));
Scene scene = new Scene(fxmlLoader.load(), 320, 240);
stage.setTitle("Hello!");
stage.setScene(scene);
```

Algunos métodos interesantes para la Scene son:

- **getWidth, getHeight**: proporciona el tamaño actual de la escena.

- **getX, getY**: obtiene las coordenadas actuales de la escena en la pantalla (haciendo referencia a su esquina superior izquierda)

- **setRoot (Parent)**: establece un nuevo administrador de diseño como nodo principal para esta escena

Un Stage puede tener múltiples Scenes, y puede cambiar entre ellas llamando a su método setScene, como veremos más adelante.

https://docs.oracle.com/javase/8/javafx/api/javafx/scene/Scene.html

10. Alerts

Un *alert*, es una alerta al usuario, normalmente para mostrar un *Warning* o un *Error* pero hay más tipos como el *information* o el *confirmation.*

Para probarlos crearemos una pequeña aplicación con un botón para cada alert y este código de ejemplo:

En la vista:

```
<Button text="Error" onAction="#mostrarAlertError"/>
<Button text="Info" onAction="#mostrarAlertInfo"/>
<Button text="Warning" onAction="#mostrarAlertWarning"/>
<Button text="Confirmation" onAction="#mostrarAlertConfirmation"/>
```

En el controlador:

```
@FXML
private void mostrarAlertError(ActionEvent event) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setHeaderText(null);
    alert.setTitle("Error");
    alert.setContentText("Error en la aplicacion");
    alert.showAndWait();
}

@FXML
private void mostrarAlertInfo(ActionEvent event) {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setHeaderText(null);
    alert.setTitle("Info");
    alert.setContentText("Informacion sobre la aplicación");
    alert.showAndWait();
}
@FXML
private void mostrarAlertWarning(ActionEvent event) {
    Alert alert = new Alert(Alert.AlertType.WARNING);
    alert.setHeaderText(null);
    alert.setTitle("Info");
    alert.setContentText("Warning en la aplica");
    alert.showAndWait();
}
@FXML
private void mostrarAlertConfirmation(ActionEvent event) {
    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
    alert.setHeaderText(null);
    alert.setTitle("Confirmacion");
    alert.setContentText("¿Deseas realmente confirmar?");
    alert.showAndWait();
}
```

11. Controles más usados en JavaFX

Hay muchos controles que se pueden usar en JavaFx podemos ver el Api para conocerlos todos.

https://docs.oracle.com/javase/8/javafx/api/toc.htm

Ahora veremos un resumen de los más usados.

11.1. Label

Se utiliza un *label* para mostrar texto en la escena. Una vez que hemos colocado el *label* dentro de un diseño, existen algunos métodos útiles en la clase Label, como getText o setText, para obtener/establecer el texto de la etiqueta.

```
@FXML
private Label label;
```

Para inicializar esta etiqueta deberemos incluirla en el controlador e inicializarla en un método o modificarla en algún método, veremos un ejemplo modificando la aplicación que se genera en intellij:

```
public class HelloController {
    @FXML
    private Label welcomeText;

    @FXML
    /*Este método ya existe en el programa de ejemplo*/
    protected void onHelloButtonClick() {
        welcomeText.setText("Si me pulsas cambio");
    }

    /*Añado este método que lo que hace es inicializar los controles
si quiero. Y le pongo al label un valor por defecto, también podría
ponerle valor por defecto en el fxml*/
    public void initialize(){
        welcomeText.setText("Hola Caracola!");
    }

}
```

11.2. Button

Si queremos utilizar un botón, lo creamos con su texto (también existen otros constructores, similares a los que podemos encontrar para la clase Label):

```
Añadimos un id: <Button text="Hello!" onAction="#onHelloButtonClick"
fx:id="btHello"/>
```

Con el id podemos acceder a los métodos del botón en el controlador.

```
@FXML
private Button btHello;
```

```
public void initialize()
{
    welcomeText.setText("Hola Caracola!");
    btHello.
}
```

```
setText(String s)                                   void
cancelButtonProperty()                   BooleanProperty
cast                                   ((SomeType) expr)
defaultButtonProperty()                  BooleanProperty
fire()                                             void
isCancelButton()                                boolean
isDefaultButton()                               boolean
setCancelButton(boolean b)                         void
setDefaultButton(boolean b)                        void
equals(Object obj)                              boolean
hashCode()                                          int
Press Intro to insert, Tabulador to replace  Next Tip
```

11.3. RadioButton

Son un conjunto de botones donde solo se puede seleccionar uno de ellos al mismo tiempo. Necesitamos definir un grupo (ToggleGroup class) y agregarle los botones de opción.

En la vista, creamos los radiobuttons, que también podrían crearse directamente en la escena mediante programación, eso lo veremos más adelante.

```
<RadioButton text="Hombre" fx:id="rbHombre"
onMouseClicked="#pulsadoHombre" />
<RadioButton text="Mujer" fx:id="rbMujer"
onMouseClicked="#pulsadoMujer" />
<RadioButton text="Otros" fx:id="rbOtros"
onMouseClicked="#pulsadoOtros" />
```

En el controlador:

```
@FXML
public RadioButton rbHombre;
public RadioButton rbMujer;
public RadioButton rbOtros;
```

```
public void initialize(){
    welcomeText.setText("Hola Caracola!");

    ToggleGroup tgRadio = new ToggleGroup(); //El toggleGroup lo
creamos para que los radiobuttons sean excluyentes
    rbHombre.setToggleGroup(tgRadio); //a todos los radiobuttons les
asignamos a toggleGroup pertenecen
    rbMujer.setToggleGroup(tgRadio);
    rbOtros.setToggleGroup(tgRadio);

}
```

```
public void pulsadoMujer(MouseEvent mouseEvent) {
    welcomeText.setText(rbMujer.getText()); //He puesto esto como
podía haber hecho cualquier otra cosa. Solo hacemos que se muestre el
texto del radiobutton pulsado en el label.
}

public void pulsadoHombre(MouseEvent mouseEvent) {
    welcomeText.setText(rbHombre.getText());
}

public void pulsadoOtros(MouseEvent mouseEvent) {
    welcomeText.setText(rbOtros.getText());
}
```

11.4. CheckBox

Un checkbox es un control que podemos marcar y desmarcar, alternativamente, cada vez que hacemos clic en él.

```
<CheckBox fx:id="cbAceptar" mnemonicParsing="false" selected="true"
text="CheckBox" onAction="#pulsadoAceptar"/>
```

En el controlador:

```
public void pulsadoAceptar(ActionEvent actionEvent) {
    if(cbAceptar.isSelected()){
        welcomeText.setText("Aceptas condiciones");
    }else{
        welcomeText.setText("No aceptas condiciones");
    }
}
```

11.5. TextFields y TextArea

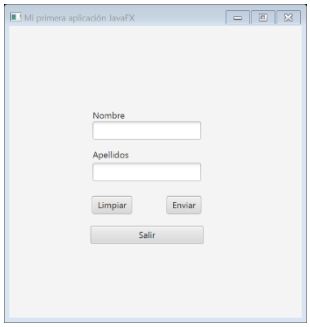
Son los controles más comunes en una aplicación junto con los *labels* y los *buttons*. El *TextField*, es para campos de texto de una sola línea, y el *TextArea*, es para campos de texto de varias líneas.

```
<TextField fx:id="txfUno" promptText="Texto por defecto"/>
```

11.5.1. Ejercicio 1. Modifica la aplicación que hemos estado haciendo, para que el txfUno alinee su texto al centro y que cuando pulses en el btHello el texto que hay escrito aparezca en el welcomeText.

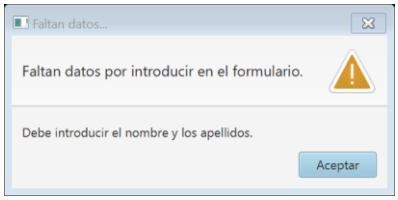
11.5.2. Ejercicio 2. Crea un TextArea de 200x200 que inicialmente esté oculto y que aparezca al hacer el click en el botón btHello.

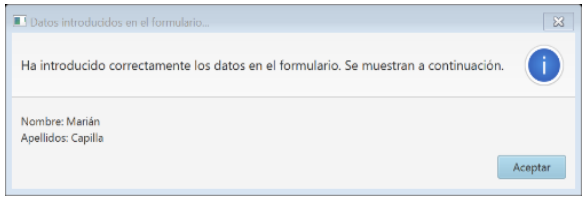
11.5.3. Ejercicio 3. Crea una aplicación con el siguiente aspecto:



La aplicación hará lo siguiente:

1. Cuando le demos a enviar, si los campos están vacíos mostrará un alert de warning diciendo que hay que rellenarlos, si están correctos, los enviará al alert y los mostrará en el mismo.





2. Cuando pulsemos en borrar, vaciará los campos.

3. Cuando pulsemos en salir, preguntará si estamos seguros de cerrar la aplicación, si es que sí la cerrará si contestamos que no, continuaremos.

11.6. Lists

Hay dos tipos de lista, las que tienen un tamaño fijo y cargan varios elementos que se ven al mismo tiempo y puedes hacer *scroll* para ver el resto de elementos Para trabajar con estas listas en JavaFX, tenemos el control ListView que utiliza una ObservableList de elementos para mostrar.

```
ObservableList<String> soList = FXCollections.observableArrayList(
"Windows", "Linux", "Mac OS X");
lList.setItems(soList);
```

Podemos definir el modelo de selección de la lista, es decir, si queremos habilitar la selección de múltiples elementos, o si solo queremos seleccionar un elemento. Podemos cambiar esta característica con estos métodos:

```
@FXML
private ListView<String> lwList;

//En la inicialización

lwList.getSelectionModel().setSelectionMode(SelectionMode.MULTIPLE);
lwList.getSelectionModel().setSelectionMode(SelectionMode.SINGLE);
```

Si queremos obtener los elementos seleccionados de la lista, tenemos que obtener el modelo de selección de la lista y luego llamar a getSelectedItem (para listas de selección única) o getSelectedItems (para listas de selección múltiple). También tenemos getSelectedIndex y getSelectedIndexes, si queremos obtener las posiciones de los elementos seleccionados, en lugar de sus valores.

```
ObservableList<String> selectIt  =
lList.getSelectionModel().getSelectedItems();
System.out.println(selectIt);
//Para más de un ítem, sino con un String sobra
```

Listas desplegables, donde sólo se muestra un elemento, y podemos elegir cualquier otro elemento extendiendo la lista. Si queremos utilizar este tipo de listas en nuestras aplicaciones JavaFX, podemos elegir entre las clases ChoiceBox y ComboBox. Las diferencias entre ellos son bastante sutiles, aunque ComboBox es más apropiado cuando trabajamos con listas grandes.

Ya sea si trabajamos con ChoiceBox o ComboBox, creamos las listas de forma muy similar a la que se muestra para ListView: necesitamos una ObservableList para configurar los elementos, y luego creamos la lista con ellos.

```
@FXML
private ComboBox<String> combList;
@FXML
private ChoiceBox<String> chbList;

//En la inicialización

chbList.setItems(soList);
combList.setItems(soList);



//Cuando queremos obtener los datos (Yo lo he hecho en el click del botón)

ObservableList<String> selectIt  =
lwList.getSelectionModel().getSelectedItems();
System.out.println(selectIt);

String selectIt2  = chbList.getSelectionModel().getSelectedItem();
System.out.println(selectIt2);
String selectIt3  = combList.getSelectionModel().getSelectedItem();
System.out.println(selectIt3);
```

11.7. Menús

Como en muchas aplicaciones de escritorio, podemos añadir un menú a nuestra aplicación JavaFX (esto no es habitual cuando estamos desarrollando una aplicación móvil). El patrón que normalmente seguimos es poner una barra de menú (con menús predeterminados dentro que podemos editar), definir las categorías (Menú) y agregar elementos de menú a las categorías.

Como puedes ver, necesitamos utilizar tres elementos diferentes:

• **MenuBar** para definir la barra donde se colocarán todos los menús y elementos del menú

• **Menu** para definir las categorías de los elementos del menú. Una categoría es un elemento que se puede mostrar, pero no tiene acción (si pulsamos sobre él no pasa más que mostrar los elementos que contiene esa categoría).

• **MenuItem** define cada elemento de nuestros menús. Si hacemos clic en un elemento, podemos definir algún código asociado a esa acción, como veremos al hablar de eventos. En el ejemplo anterior hemos definido un elemento de menú llamado "Cerrar" dentro del menú Archivo.

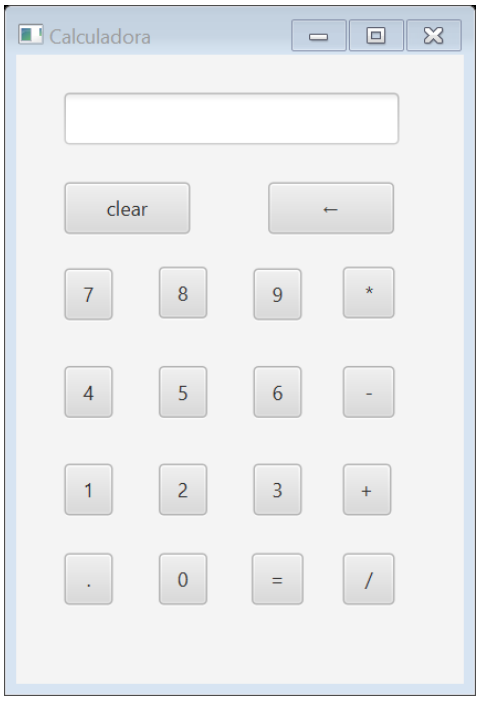
- También hay algunos subtipos de **MenuItem**, como **CheckMenuItem** (elementos que se pueden marcar o desmarcar, como casillas de verificación) o **RadioMenuItem** (grupos de elementos donde solo uno de ellos se puede marcar al mismo tiempo, como

botones de opción). También podemos usar un **SeparatorMenuItem** para crear una línea de separación entre grupos de elementos del menú.

Código en el fxml

```
<MenuBar>
    <Menu mnemonicParsing="false" text="File">
        <MenuItem mnemonicParsing="false" text="Close"
fx:id="menuClose" onAction="#cerrarApp"/>
    </Menu>
    <Menu mnemonicParsing="false" text="Edit">
        <MenuItem mnemonicParsing="false" text="Delete"/>
    </Menu>
    <Menu mnemonicParsing="false" text="Help">
        <MenuItem mnemonicParsing="false" text="About"/>
    </Menu>
</MenuBar>
</MenuBar>
```

Código en el controlador

```
public void cerrarApp(ActionEvent actionEvent) {
    System.exit(0);
}
```

11.7.1. Crea una calculadora con este aspecto. Haz que realice las operaciones básicas. Suma, resta, multiplicación y división, que pueda borrar por dígitos y limpiar el TextField, y que actúe con números decimales.